

Estruturas de Dados

No terceiro tópico vamos realizar a [análise de complexidade](#) de estrutura de dados fundamentais. Também serão avaliadas estruturas de árvores binárias, árvores auto-balanceadas e [grafos](#).



Objetivo

Os alunos deverão avaliar a complexidade de estruturas de dados em árvore e [grafos](#).

- Avaliar a complexidade de **estruturas de dados fundamentais**: pilha, fila, listas encadeadas, string e hash;
- Calcular a complexidade de espaço e tempo em estruturas de árvores auto-balanceadas;
- Fixar os conceitos de **grafos** e árvores.



Referência para Estudo

CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 4. ed. Rio de Janeiro: LTC, 2024. [Parte III](#), [Parte V](#) e [Parte VI](#)

CORMEN, Thomas H. Desmistificando algoritmos. Rio de Janeiro: Elsevier, 2014. [Capítulo 5](#)

DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos H; VAZIRANI, Umesh Virkumar. Algoritmos. São Paulo: McGraw-Hill, 2009. [Capítulo 4](#) e [Capítulo 5](#)



[Estruturas de Dados Fundamentais](#) ○

✓ Done



[Estruturas de Dados Fundamentais - exercícios](#)



1. Considere a sequência de chaves Q U E S T A O F C I L e a codificação A = 0, B = 1, C = 2, etc.
 - a) Desenhe o conteúdo da tabela hash resultante da inserção dos registros com essas chaves nessa ordem em uma tabela inicialmente vazia de tamanho 7 usando listas encadeadas. Use a função hash $h(k) = k \bmod 7$ onde k é a codificação da letra.
 - b) Desenhe o conteúdo da tabela hash resultante da inserção dos registros com essas chaves nessa ordem em uma tabela inicialmente vazia de tamanho 13 usando endereçamento aberto e hash linear para tratar colisões. Use a função hash $h(k) = k \bmod 13$ onde k é a codificação da letra.
2. Considere a implementação de uma tabela Hash de tamanho M=11, com endereçamento aberto utilizando a função $k \bmod M$. Responda as seguintes questões:
 - a) Mostre a configuração da tabela após a inserção dos registros com as chaves: 4, 17, 13, 35, 25, 11, 2, 10, 32.
 - b) Mostre a configuração da tabela após a remoção dos registros com as chaves: 25, 11.
 - c) Mostre a configuração da tabela após a inserção dos registros com as chaves: 40, 3.



[SHA256: o que é, como funciona e para que serve? \(TOTVS\)](#)



[Hash para busca de arquivos duplicados \(rmlint\)](#)

-a --algorithm=name (default: blake2b):

Choose the algorithm to use for finding duplicate files. The algorithm can be either **paranoid** (byte-by-byte file comparison) or use one of several file hash algorithms to identify duplicates. The following hash families are available (in approximate descending order of cryptographic strength):

- sha3, blake,
- sha,
- highway, md
- metro, murmur, xxhash

The weaker hash functions still offer excellent distribution properties, but are potentially more vulnerable to *malicious* crafting of duplicate files.

The full list of hash functions (in decreasing order of checksum length) is:

- 512-bit: **blake2b**, **blake2bp**, **sha3-512**, **sha512**

?

- 384-bit: **sha3-384**,
- 256-bit: **blake2s**, **blake2sp**, **sha3-256**, **sha256**, **highway256**, **metro256**, **metrocrc256**
- 160-bit: **sha1**
- 128-bit: **md5**, **murmur**, **metro**, **metrocrc**
- 64-bit: **highway64**, **xxhash**.

The use of 64-bit hash length for detecting duplicate files is not recommended, due to the probability of a random hash collision.

-p --paranoid / -P --less-paranoid (default):

Increase or decrease the paranoia of **rmlint**'s duplicate algorithm. Use **-p** if you want byte-by-byte comparison without any hashing.

- **-p** is equivalent to **--algorithm=paranoid**
- **-P** is equivalent to **--algorithm=highway256**
- **-PP** is equivalent to **--algorithm=metro256**
- **-PPP** is equivalent to **--algorithm=metro**



[Árvores Binárias e auto-balanceadas](#)

✓ Done



[Estruturas de dados em C++, Java e Python](#)

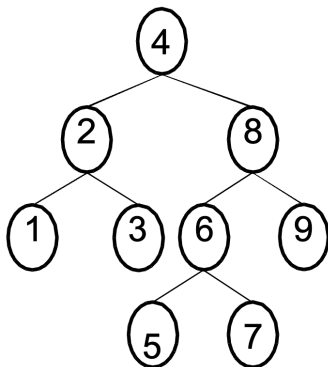


[Árvores Binárias e auto-balanceadas - exercícios](#)

✓ Done

Implemente um algoritmo que visite todos os nós de uma árvore binária com uma busca em profundidade em-ordem, pré-ordem e pós-ordem.

- Exemplo, para a árvore abaixo a saída deve ser:
 - Pré-ordem: 4, 2, 1, 3, 8, 6, 5, 7, 9
 - Em-ordem: 1, 2, 3, 4, 5, 6, 7, 8, 9
 - Pós-ordem: 1, 3, 2, 5, 7, 6, 9, 8, 4



Acessível com
VLibras



[Árvores 2-3, B e B+](#)

✓ Done



[Árvores 2-3, B e B+ - exercícios](#)

✓ Done

1. Quantas chaves, no máximo, pode conter uma árvore 2-3 de altura 2? Qual o número mínimo de chaves em uma árvore 2-3 de altura 2? Desenhe exemplos dessas árvores.
2. Desenhe a árvore 2-3 que resulta da inserção das chaves [10, 1, 20, 30, 18, 25, 24, 11, 3], nesta ordem, em uma árvore inicialmente vazia.
3. Os dados numa árvore 2-3 são mantidos ordenados? Justifique.
4. Existe alguma relação entre a árvore Rubro-Negra e a árvore 2-3? Seria possível representar qualquer árvore Rubro-Negra em uma árvore 2-3?
5. Pesquise sobre árvores 2-3-4 (também chamada de 2-4). Verifique qual a relação entre ela e a árvore Rubro-Negra. Qual vantagem e desvantagem em relação à árvore 2-3?
6. Mostre todas as árvores B válidas com grau mínimo 2 (M=4) que incluem as chaves 1, 2, 3, 4 e 5.
7. Mostre o resultado da inserção (nesta ordem) das chaves 16, 29, 27, 21, 13, 22, 18, 30, 32, 33, 23, 28, 24, 26, 11, 12, 34, 35, 14, 36, 15 em uma árvore B de grau 3 (M=6). Mostre a configuração da árvore após cada inserção e imediatamente antes de cada split.
8. Quantas leituras são feitas no pior caso para buscar um elemento em uma árvore B. Quantas leituras e escritas são necessárias para inserir um elemento, no pior caso?



[Grafos](#)

✓ Done

[Processamento de Imagens em Grafos](#) ○[Floresta de Caminhos Ótimos](#) ○

Imagens e **Grafos**

Abaixo seguem alguns links com materiais disponíveis em disciplinas sobre processamento de imagens usando [grafos](#):

- [Página de disciplina na USP - Processamento de Imagens usando Grafos \(MAC6903\)](#)
- [Página de disciplina na UNICAMP - Processamento de Imagens usando Grafos \(MO815/MC919\)](#)
- [Apresentação sobre IFT do prof. Alexandre Falcão](#)
- Vídeo Aula sobre OPF do prof. João Paulo Papa:

[Avaliação Tópico 3](#)

✓ Done ▾

Questões finais do tópico 3 (AT₃)

IMPORTANTE: Limite de tempo de **100 minutos** (1h40m)

[Trabalho Prático 3 \(TP-3\)](#)

✓ Done ▾

Trabalho Prático 3 – Autocompletar com Árvore Trie

O objetivo deste trabalho é permitir ao aluno implementar um algoritmo para previsão de palavras a partir do início da escrita para auxílio na tarefa de autocompletar.

Descrição do Trabalho

O trabalho consiste em implementar um algoritmo para auxílio na tarefa de autocompletar usando a estrutura de dados chamado Árvore Trie. O programa deve ler um dicionário e montar a árvore. Após lido o dicionário, o programa deve permitir que o usuário escreva o início de uma palavra. A partir do início, o programa deve sugerir quais palavras o usuário pretende escrever.

Você pode utilizar como dicionário um conjunto de palavras reservadas de uma linguagem de programação. Desta forma, o programa pode se comportar como uma IDE que auxilia o programador a digitar mais rápido. Também é possível usar como dicionário os nomes de cidade, nomes de medicamentos ou outros para ser utilizado em um programa que busca informações em um campo texto (ou lista suspensa).

A saída do programa é uma lista (em tela) de palavras do dicionário que iniciam pelas letras digitadas pelo usuário.

Exemplo

Seu programa deve ler letra a letra e exibir as possíveis palavras que o usuário queira digitar. A seguir um exemplo:

```
> minha_frutaria
```

Escolha uma fruta ou legume: ab<enter>

Sugestões:

abacaxi
abacate
abóbora
abobrinha

[Trabalho Prático 3 \(TP-3 alternativo\)](#)

✓ Done ▾

Trabalho Prático 3 – Possíveis soluções para jogo **Termo**

O objetivo deste trabalho é permitir ao aluno implementar um algoritmo para previsão de palavras para auxiliar na resolução do jogo **Termo**.

Descrição do Trabalho

O trabalho consiste em implementar e um algoritmo para auxílio na solução do jogo Termo usando a estrutura de dados chamado Árvore Trie. O programa deve ler um dicionário e montar a árvore. Após lido o dicionário, o programa deve permitir que o usuário escreva uma palavra de teste e as cores retornadas pelo website (**A**marelo, **V**erde ou **P**reto). A partir da palavra de teste (chute) e da lista de cores, o programa deve sugerir as possíveis palavras usando a árvore Trie gerada a partir do dicionário.



Adicionalmente, você também poderá utilizar as respostas anteriores para melhorar a resposta do programa. Também é possível gerar uma versão do Termo offline para fazer o computador "jogar sozinho" mostrando as tentativas e possíveis soluções de forma iterativa.

Exemplo

Seu programa deve uma palavra e um código de cores para tentar descobrir a "palavra chave":

```
> termo_solver
```

Chute #1: TERMO
resposta: A PPPP

Sugestões de Palavras
(total de 182)
FALTA
PAUTA
CANTA
DATAS
LATAS
JANTA

Chute #2: PAUTA
resposta: V V U V V

Sugestões de Palavras
(total de 1)
PASTA

Chute #3: PASTA
resposta: V V V V V (resposta correta)

